

**МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ**

Національний Технічний Університет України

«Київський Політехнічний Інститут  
імені Ігоря Сікорського»

**КАТЕДРА ОБЧИСЛЮВАЛЬНОЇ ТЕХНІКИ**

# **Теорія ймовірностей та математична статистика**

## **Лабораторна робота №5**

«Дослідження неперервних та дискретних процесів Маркова»

**Виконав:**

студент групи ІО-32

Крадожон М.

**Перевірів(-ла):**

Марковський О.

# Лабораторна робота №5

## Відомості

### Тема

Дослідження нерервних та дискретних процесів Маркова

### Варіант

11

Число станів: 7

Тип процесу: **Дискретний (Д)**

## Хід роботи

### Теоретична частина

#### 1. Опис моделі

Позначимо стани як:  $S = \{0, 1, 2, 3, 4, 5, 6\}$

Припустимо, що маємо **матрицю переходів**  $P = [p_{ij}]$ , де  $p_{ij}$  - ймовірність переходу зі стану  $i$  в стан  $j$ .

Від вас не надана конкретна матриця переходів. Щоб вирішити задачу, візьмемо **прикладову матрицю**, яку ви зможете замінити на вашу:

$$P = \begin{bmatrix} 0.1 & 0.4 & 0.1 & 0.2 & 0.1 & 0.05 & 0.05 \\ 0.2 & 0.3 & 0.2 & 0.1 & 0.1 & 0.05 & 0.05 \\ 0.1 & 0.1 & 0.5 & 0.1 & 0.1 & 0.05 & 0.05 \\ 0.1 & 0.1 & 0.1 & 0.4 & 0.2 & 0.05 & 0.05 \\ 0.1 & 0.1 & 0.1 & 0.1 & 0.4 & 0.1 & 0.1 \\ 0.05 & 0.05 & 0.05 & 0.1 & 0.1 & 0.6 & 0.05 \\ 0.1 & 0.05 & 0.05 & 0.05 & 0.05 & 0.1 & 0.6 \end{bmatrix}$$

## 2. Система рівнянь

Стационарні ймовірності - вектор  $\pi = [\pi_0, \pi_1, \dots, \pi_6]$ , що задовольняє:

$$\begin{cases} \pi P = \pi \\ \sum_{i=0}^6 \pi_i = 1 \end{cases}$$

Це означає, що  $\pi$  - власний вектор матриці  $P$ , що відповідає власному значенню 1.

# Практична частина на Python

```
import numpy as np
import random

P = np.array([
    [0.1, 0.4, 0.1, 0.2, 0.1, 0.05, 0.05],
    [0.2, 0.3, 0.2, 0.1, 0.1, 0.05, 0.05],
    [0.1, 0.1, 0.5, 0.1, 0.1, 0.05, 0.05],
    [0.1, 0.1, 0.1, 0.4, 0.2, 0.05, 0.05],
    [0.1, 0.1, 0.1, 0.1, 0.4, 0.1, 0.1],
    [0.05, 0.05, 0.05, 0.1, 0.1, 0.6, 0.05],
    [0.1, 0.05, 0.05, 0.05, 0.05, 0.1, 0.6]
])

n_states = 7
n_steps = 1000
state_counts = np.zeros(n_states)

# Початковий стан
current_state = random.randint(0, n_states - 1)

for _ in range(n_steps):
    state_counts[current_state] += 1
    current_state = np.random.choice(n_states, p=P[current_state])

experimental_probs = state_counts / n_steps
print("Експериментальні стаціонарні ймовірності:")
print(experimental_probs)

eigvals, eigvecs = np.linalg.eig(P.T)
stationary = eigvecs[:, np.isclose(eigvals, 1)]
stationary = stationary[:, 0]
stationary = stationary / stationary.sum()

print("\nТеоретичні стаціонарні ймовірності:")
print(np.real(stationary))
```

## Результати

*Експериментальні стаціонарні ймовірності:*

*[0.113 0.14 0.151 0.145 0.15 0.17 0.131]*

*Теоретичні стаціонарні ймовірності:*

*[0.10771655 0.14846102 0.16883325 0.14907865 0.15498753 0.14259105  
0.12833195]*